

AWS ECS (Basic)

Elastic Container Service

Concept Overview:

Introduction of ECS	1
ECS Key Concepts	2
ECS Launch Types	3
ECS Components	4
ECS Autoscaling	5
ECS Networking	6
ECS Monitoring	7
ECS Deployment Strategies	8
Launch First ECS - Lab	9
Diff between traditional deployment & ECS	10

Introduction of ECS:

Amazon ECS is a fully managed container orchestration service that helps you easily deploy, manage, and scale containerized applications.

It deeply integrates with the rest of the AWS platform to provide a secure and easy-to-use solution for running container workloads in the cloud and now on your infrastructure with Amazon ECS Anywhere.

The reason ECS (and container orchestration in general) matters is best understood by first looking at how applications used to be deployed, what pain that caused, and then seeing exactly which of those pains ECS removes.

That is the structure this guide follows: traditional deployment first, then the problems, then the solution, then a deep technical walkthrough of every major ECS building block, launch types, components, scaling, networking, monitoring, security, deployment strategies, and storage, finishing with a practical roadmap for what to learn next.

ECS Key Concepts:

- Containers & Images: Container is a lightweight, isolated unit of software that packages application code together with everything it needs to run (runtime, libraries, config). A Docker image is the immutable template a container is launched from, the same image produces identical containers wherever it runs.

- Cluster: Cluster is a logical grouping of compute capacity (EC2 instances or serverless Fargate capacity) that ECS uses to run containers. It's the boundary within which tasks and services are scheduled.

- Task Definition: A JSON blueprint describing one or more containers that should run together, including which image to use, CPU/memory limits, port mappings, environment variables, IAM role, logging configuration, and storage/volume mounts. It's analogous to a docker-compose file, but native to ECS

- Task: Task is a running instance of a task definition, the actual, live set of containers executing on the cluster right now.

ECS Key Concepts:

- Service: Service manages a desired number of long-running tasks. It keeps that count alive (relaunching failed tasks), integrates with a load balancer, and drives rolling deployments when the task definition changes.

- Container Agent: On EC2-backed clusters, the ECS Container Agent runs on each instance, communicating with the ECS control plane to start/stop tasks and report instance/task status.

ECS Launch Types:

1. EC2 Launch Type
2. Fargate Launch Type
3. Fargate Spot
4. External / ECS Anywhere

1. EC2 Launch Type: You provision and manage your own EC2 instances. Instance type, OS patching, capacity via Auto Scaling Groups. While ECS schedules containers onto them efficiently. You get more control but also more responsibility, managing scaling at both the fleet and container level. Best for specialized hardware needs or tighter cost control at scale.

2. Fargate Launch Type: Serverless compute for containers. You just specify CPU/memory per task, and AWS handles the rest, provisioning, patching, and isolation. Best for most workloads and teams wanting minimal operational overhead.

3. Fargate Spot: Fargate on spare AWS capacity at up to ~70% lower cost, reclaimable with a two-minute warning. Best for fault-tolerant workloads like batch jobs or dev/test environments.

4. External / ECS Anywhere: Extends ECS to manage containers on your own on-premises infrastructure, useful for hybrid cloud migrations.

ECS Components:

1. Cluster: The top-level namespace of compute capacity EC2 instances, Fargate, or both where services and tasks run.

2. Task Definition: An immutable, versioned blueprint describing the containers to run: image, CPU/memory, ports, environment variables, IAM roles, logging, and storage. Updating it creates a new revision rather than modifying the old one, which is what makes safe rollbacks possible.

3. Task: A live, running instance of a task definition. Each task gets its own network interface and IP address (with awsvpc mode).

4. Service: Keeps a desired number of tasks running at all times, relaunching failed ones, registering/deregistering with a load balancer, and driving deployments when the task definition changes.

5. Container Instance (EC2 launch type only): An EC2 instance registered to a cluster, running the ECS Container Agent, capable of hosting one or more tasks.

ECS Components:

6. Capacity Provider: Defines how a cluster's compute is sourced and scaled, Fargate, Fargate Spot, or an EC2 Auto Scaling Group, and can mix strategies for cost and resilience.

7. ECS Scheduler: The control-plane logic that places tasks efficiently across the cluster, continuously working to keep the running state matched to each service's desired state.

ECS Autoscaling:

ECS scaling operates on two independent layers, and it's important to understand both.

Layer 1 - Service Auto Scaling (task-level):
Adjusts the number of tasks a service runs, using AWS Application Auto Scaling. Common policies:

- Target Tracking: Keep a metric (e.g. average CPU at 50%) near a target; ECS automatically adds/removes tasks to hold that target. This is the most common and recommended approach.

- Step Scaling: Scale by a defined amount based on how far a CloudWatch alarm threshold is breached (e.g. +2 tasks if CPU > 80%, +4 tasks if CPU > 95%).

- Scheduled Scaling: scale to a known capacity at a known time (e.g. scale up before a predictable daily traffic peak).

ECS Autoscaling:

Layer 2 - Cluster / Capacity Auto Scaling (infrastructure-level, EC2 launch type only):

When using the EC2 launch type, the tasks need somewhere to run — so the underlying EC2 fleet must also scale. Cluster Auto Scaling, via Capacity Providers, monitors cluster capacity utilization and grows/shrinks the Auto Scaling Group backing the cluster so there's always enough room to place tasks (and not excessive idle capacity wasting cost).

ECS Networking:

Networking is one of the more conceptually dense areas of ECS. The key building blocks are network modes, VPC layout, load balancing, and service discovery.

Network Modes:

- awsvpc: Each task gets its own elastic network interface (ENI) and private IP address inside the VPC, the modern default, required for Fargate. Enables per-task security groups.

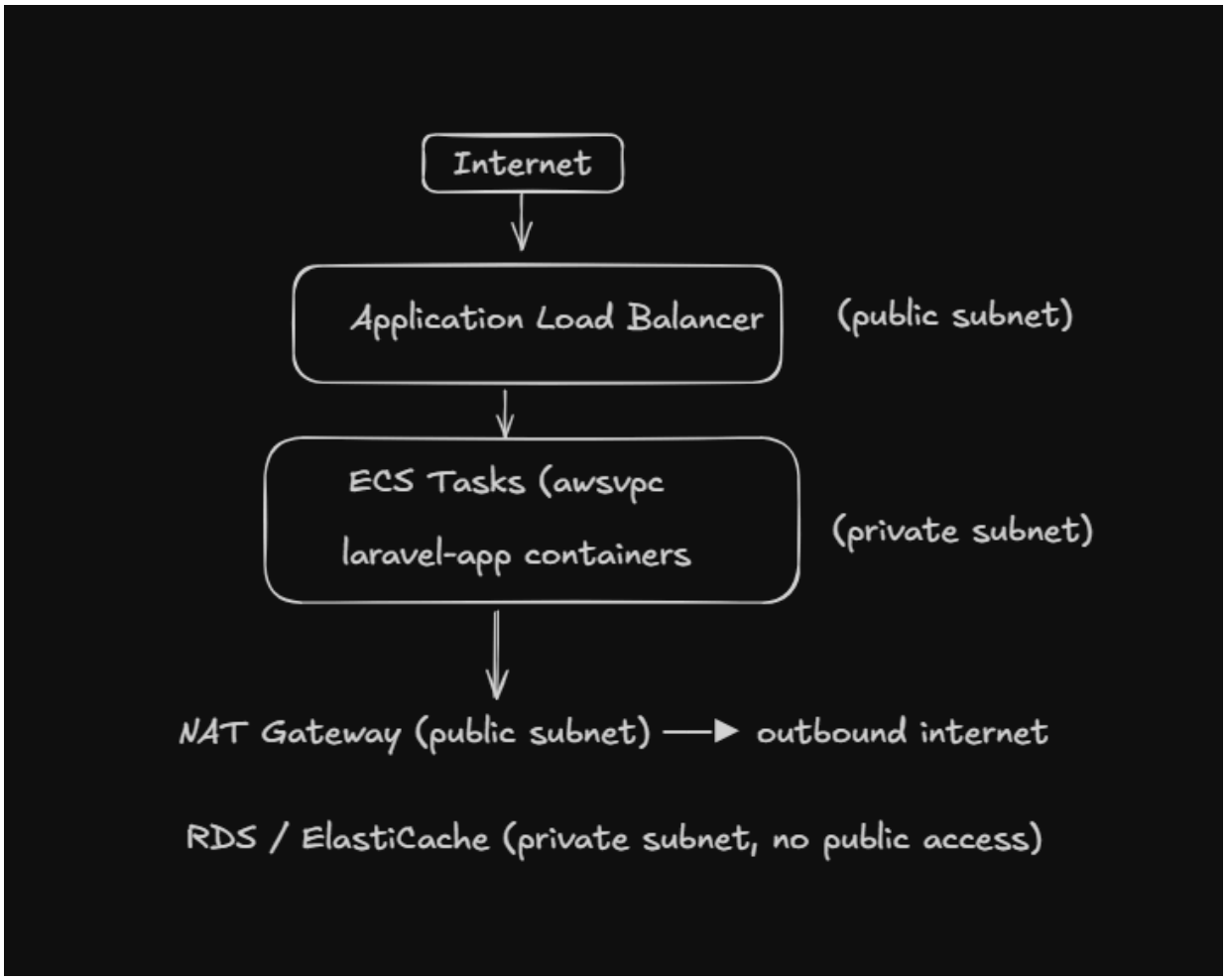
- bridge: Docker's default bridge networking on the host (EC2 launch type only) — containers share the host's network stack via port mapping.

- host: Container uses the host's network namespace directly (EC2 only) — no port mapping isolation, higher performance, less isolation.

- none: No external networking for the container.

ECS Networking:

VPC Layout:
A typical production ECS setup places tasks in private subnets (no direct internet inbound), with a NAT Gateway in a public subnet providing outbound internet access (e.g. to pull images, call external APIs), and a public-facing load balancer in public subnets routing traffic in.



ECS Networking:

Load Balancing:
ECS integrates natively with the Application Load Balancer (ALB) (HTTP/HTTPS, path- and host-based routing — the usual choice for web apps) and the Network Load Balancer (NLB) (for raw TCP/UDP, extreme performance, static IPs). The ECS service automatically registers each new task's IP into the load balancer's target group and deregisters it on shutdown, health-checking continuously.

Security Groups:
With awsvpc mode, security groups attach directly to each task's ENI. Common pattern: the ALB's security group allows inbound 443/80 fromthe internet; the task's security group allows inbound only from the ALB's security group — nothing else can reach the containers directly.

Service Discovery / Service Connect:
For service-to-service communication in a microservices setup, ECS offers Service Discovery (via Route 53 private DNS namespaces, giving each service a resolvable internal DNS name) and the newer ECS Service Connect, which adds a managed proxy layer for simpler, more resilient internal service-to-service traffic with built-in metrics.

ECS Monitoring:

Observability in ECS centers on three main AWS-native tools, plus health checks at multiple levels.

Amazon CloudWatch Metrics:
ECS automatically publishes cluster- and service-level metrics, CPU utilization, memory utilization, running/pending/desired task counts — to CloudWatch. These metrics are what Service Auto Scaling policies typically watch.

CloudWatch Container Insights:
An enhanced monitoring layer providing detailed per-task and per-container CPU, memory, network, and storage metrics, along with pre-built dashboards, making it far easier to spot which specific task or container is misbehaving versus just cluster-wide averages.

CloudWatch Logs :
Using the awslogs log driver in the task definition, container stdout/stderr is streamed directly to a CloudWatch Logs group, centralizing logs from every task instead of having to SSH into individual instances.

ECS Monitoring:

Health Checks (multiple layers):

- Container-level health check: defined in the task definition (a command run inside the container); if it fails repeatedly, ECS stops and replaces the task.

- Load balancer health check: the ALB/NLB target group periodically hits a defined path (e.g. /health); unhealthy targets stop receiving traffic and, combined with ECS, get replaced.

- ECS service-level monitoring: the scheduler itself continuously reconciles running task count against desired count, replacing anything that has stopped.

AWS X-Ray:
For distributed tracing across microservices — following a single request as it hops between multiple ECS services — X-Ray can be integrated to visualize latency and pinpoint bottlenecks across service boundaries.

ECS Deployment Strategies :

ECS supports multiple deployment strategies for safely rolling out a new task definition revision.

1. Rolling Update (default):
Gradually replaces old tasks with new ones, controlled by minimum healthy percent and maximum percent settings. Simple and built-in, but old and new versions briefly serve traffic together during rollout.

2. Blue/Green Deployment (via CodeDeploy):
A full new task set launches alongside the current one, gets validated separately, then traffic cuts over. Rollback is near-instant if something goes wrong, at the cost of more setup.

3. Canary Deployment:
A blue/green variant that shifts a small percentage of traffic to the new version first, then gradually increases it, minimizing the blast radius of a bad release.

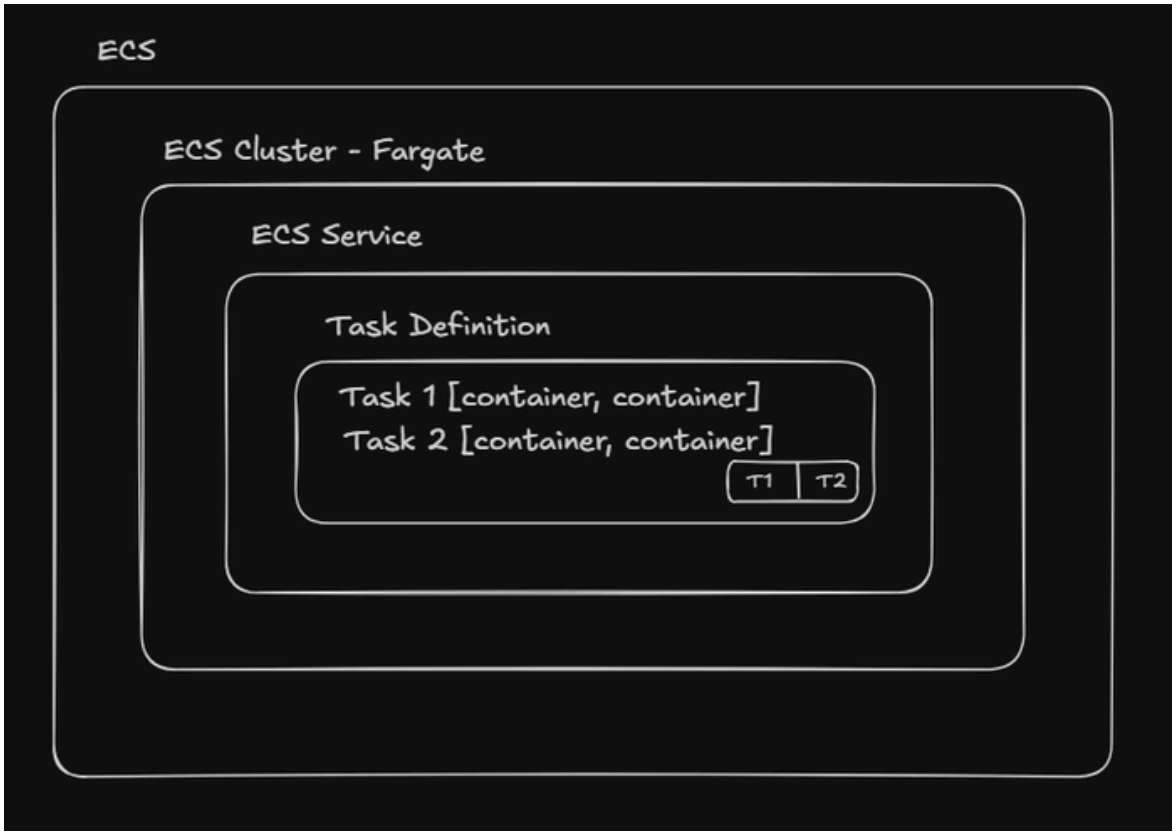
ECS Deployment Strategies :

Deployment Circuit Breaker
ECS can automatically detect a failing deployment (tasks repeatedly failing to stabilize) and automatically roll back to the previous task definition revision without manual intervention an important safety net for CI/CD pipelines.

CI/CD Integration
ECS deployments are commonly automated through AWS CodePipeline + CodeBuild + CodeDeploy, or through third-party CI tools (GitHub Actions, GitLab CI, Jenkins) that build a Docker image, push it to ECR, register a new task definition revision, and call ecs update-service to trigger the rollout.

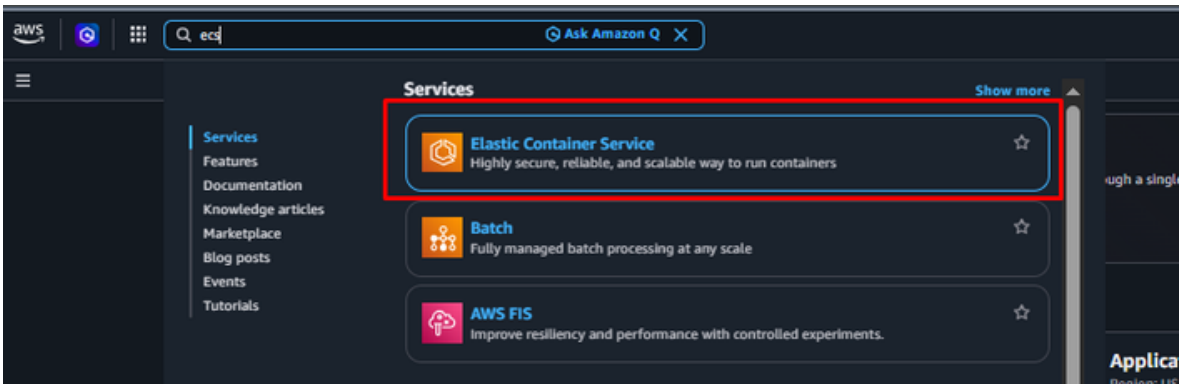
Launch First ECS - Lab:

Before jump to lab, create a simple diagram which i following to create ECS.



Step 1:

Search for ECS, Click on it and open the page.

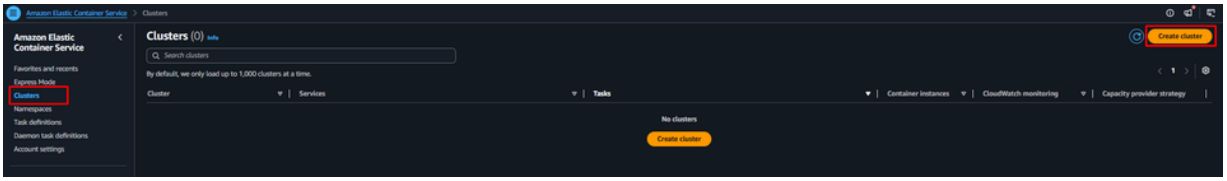


Launch First ECS - Lab:

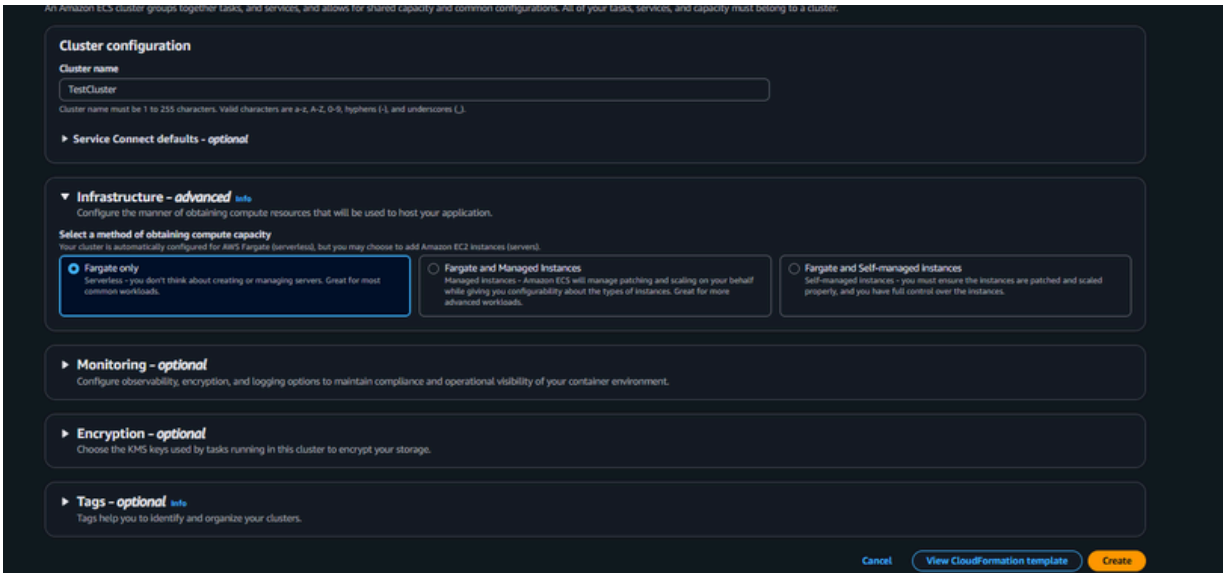
Step 2:

Create a cluster first.

To create a cluster click on cluster menu and click create cluster button.



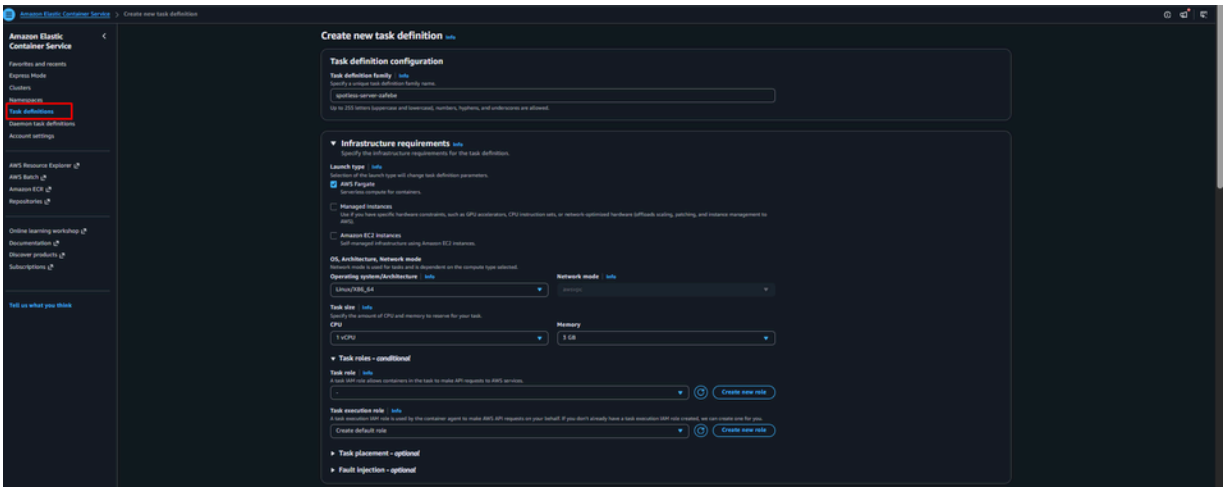
Enter a name and set other option based on your need and create.



Launch First ECS - Lab:

Step 3:

Now create a Task Definition.



Step 4:

Create service under the cluster. With necessary network configuration.

Note: Using an AWS Academy sandbox account, so IAM permissions are restricted, couldn't run ecs:CreateCluster to complete the live ECS deployment step :(

Diff between traditional deployment & ECS:

Traditional Deployment:

Servers are provisioned and configured manually, with code deployed directly onto them. Scaling means manually adding servers, and failures require human intervention to detect and fix. Configuration drifts over time, and deployments risk downtime since updates happen directly on live servers.

ECS:

ECS packages applications as containers and automates their management. You define a desired state, and ECS handles scaling, failure recovery, and deployments automatically using versioned task definitions, eliminating manual server work and configuration drift, while enabling safer rollouts with rolling updates and easy rollback.

Thank You

Stay Connect:

/in/alamgirweb11

/alamgirweb11

/alamgir.xyz